

100 MCQs with Answers: Linked Lists

Below are **100 multiple-choice questions** covering **Singly Linked Lists**, memory representation, traversal, searching, insertion, deletion, linked Stack and Queue, header nodes, Doubly Linked Lists, Circular Linked Lists, algorithms, and complexity analysis.

A. Singly Linked Lists — Basics & Memory Representation

1. What is a linked list?

- A. A collection of elements stored in contiguous memory
- B. A collection of nodes connected using links
- C. A collection of arrays
- D. A collection of functions

Answer: B

2. A node in a singly linked list generally contains:

- A. Data only
- B. Link only
- C. Data and a pointer/link to the next node
- D. Two data fields

Answer: C

3. The link field of the last node in a singly linked list contains:

- A. Address of the first node
- B. Address of the previous node
- C. NULL
- D. Zero always

Answer: C

4. The pointer that points to the first node is commonly called:

- A. Rear
- B. Head
- C. Tail
- D. Top

Answer: B

5. In a linked list, nodes are generally stored:

- A. Only in consecutive memory locations
- B. In arbitrary memory locations
- C. Only in stack memory
- D. Only in registers

Answer: B

6. Which data structure is commonly used to implement a linked list?

- A. Array
- B. Structure/record
- C. Stack only
- D. Queue only

Answer: B

7. The main advantage of a linked list over an array is:

- A. Faster random access
- B. Dynamic size
- C. Less pointer manipulation
- D. Better cache locality

Answer: B

8. Which operation is generally inefficient in a singly linked list?

- A. Insertion at beginning
- B. Deletion at beginning
- C. Random access by index
- D. Traversal

Answer: C

9. The time complexity of accessing the kth element of a singly linked list is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

10. Which pointer is required to traverse a singly linked list?

- A. Current pointer
- B. Previous pointer only
- C. Rear pointer only
- D. Stack pointer

Answer: A

11. If `head == NULL`, the linked list is:

- A. Full
- B. Empty
- C. Circular
- D. Doubly linked

Answer: B

12. A linked list is called dynamic because:

- A. Its elements are sorted automatically
- B. Its size can change during execution
- C. It uses dynamic programming
- D. It is always stored in cache

Answer: B

13. Which operation does not require shifting existing elements in a linked list?

- A. Insertion
- B. Deletion
- C. Both A and B
- D. Searching

Answer: C

14. A pointer in a linked-list node stores:

- A. Data value only
- B. Address/reference of another node
- C. Size of the list only
- D. Index of the node

Answer: B

15. Which memory allocation technique is commonly associated with dynamically created linked-list nodes?

- A. Static allocation
- B. Dynamic allocation
- C. Compile-time allocation only
- D. Register allocation

Answer: B

16. If a singly linked list contains n nodes, the number of links between nodes is:

- A. n
- B. $n + 1$
- C. $n - 1$
- D. $2n$

Answer: C

17. A singly linked list with one node has:

- A. One data field and one NULL link
- B. Two NULL links
- C. No link field
- D. Two data fields

Answer: A

18. Which statement is true about linked-list memory?

- A. Nodes must be contiguous
- B. Nodes can be scattered in memory
- C. Nodes cannot be dynamically allocated
- D. Nodes must be sorted by address

Answer: B

19. Compared with arrays, linked lists generally require:

- A. Extra memory for links
- B. Less memory always
- C. No memory for pointers
- D. Fixed memory

Answer: A

20. The head pointer of a non-empty linked list contains:

- A. Data of the first node
- B. Address of the first node
- C. Address of the last node only
- D. Number of nodes

Answer: B

B. Traversal and Searching

21. Traversal of a singly linked list means:

- A. Sorting all nodes
- B. Visiting nodes sequentially
- C. Reversing the list
- D. Deleting nodes

Answer: B

22. The time complexity of traversing n nodes is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

23. A typical traversal begins with:

- A. `current = NULL`
- B. `current = head`
- C. `current = tail`
- D. `current = rear`

Answer: B

24. During traversal, the current pointer is usually updated as:

- A. `current = NULL`
- B. `current = head`
- C. `current = current->next`
- D. `current = current->previous`

Answer: C

25. A typical traversal terminates when:

- A. `current == head`
- B. `current == NULL`
- C. `current != NULL`
- D. `current == rear`

Answer: B

26. Linear search in a singly linked list has worst-case complexity:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

27. The best-case time complexity of searching for an element in a linked list is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: A

28. In the worst case, searching an element requires examining:

- A. First node only
- B. Half the nodes
- C. All n nodes
- D. No nodes

Answer: C

29. Binary search is not directly efficient on a standard linked list because:

- A. Nodes cannot contain data
- B. Random access is not available
- C. Links cannot be followed
- D. Lists cannot be sorted

Answer: B

30. If an element is found at the first node, the search complexity is:

- A. $O(n)$
- B. $O(n^2)$
- C. $O(1)$
- D. $O(\log n)$

Answer: C

C. Insertion in Singly Linked Lists

31. Insertion at the beginning of a singly linked list generally takes:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Answer: C

32. To insert a new node at the beginning, its next pointer should point to:

- A. NULL
- B. Tail
- C. Current head
- D. Previous node

Answer: C

33. After inserting a node at the beginning, the head should point to:

- A. Old head
- B. New node
- C. Last node
- D. NULL

Answer: B

34. Which sequence correctly inserts node `N` at the beginning?

- A. `N->next = head; head = N`
- B. `head = N; N->next = NULL`
- C. `N->next = NULL; head = N`
- D. `head->next = N; head = NULL`

Answer: A

35. Inserting after a known node in a singly linked list takes:

- A. $O(1)$
- B. $O(n^2)$
- C. $O(\log n)$
- D. $O(n)$ always

Answer: A

36. To insert node `N` after node `P`, the correct link operation is:

- A. `N->next = P`
- B. `N->next = P->next; P->next = N`
- C. `P->next = NULL`
- D. `P = N->next`

Answer: B

37. Inserting at the end of a singly linked list takes $O(1)$ if:

- A. The list is sorted
- B. A tail pointer is maintained
- C. The list has one node
- D. The list is circular

Answer: B

38. Without a tail pointer, insertion at the end generally takes:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

39. If a singly linked list contains n nodes, insertion at position 1 takes:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Answer: B

40. When inserting a node between two nodes, the links that normally need modification are:

- A. Only the new node's link
- B. Only the previous node's link
- C. New node's link and previous node's link
- D. All links in the list

Answer: C

D. Deletion from Singly Linked Lists

41. Deletion from the beginning of a singly linked list takes:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Answer: B

42. To delete the first node, the head should be changed to:

- A. NULL always
- B. `head->next`
- C. `head->previous`
- D. Tail

Answer: B

43. If a list has only one node and that node is deleted, the head becomes:

- A. Tail
- B. Previous node
- C. NULL
- D. Same node

Answer: C

44. Deleting a node after a known predecessor takes:

- A. $O(1)$
- B. $O(n)$

- C. $O(n^2)$
- D. $O(\log n)$

Answer: A

45. To delete node X after node P, a common operation is:

- A. `P->next = X`
- B. `P->next = X->next`
- C. `X->next = P`
- D. `P = X->next`

Answer: B

46. Deleting the last node in a singly linked list without a tail/previous pointer generally requires:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

47. Why is deleting the last node difficult in a singly linked list?

- A. Data cannot be accessed
- B. We need to find its predecessor
- C. Last node has two pointers
- D. Head cannot be modified

Answer: B

48. Before deleting a node, we should generally:

- A. Lose its address
- B. Adjust the required links
- C. Delete the head immediately
- D. Sort the list

Answer: B

49. Which operation may require finding the predecessor of a node?

- A. Delete a node from the middle
- B. Insert at beginning

- C. Traverse
- D. Read head

Answer: A

50. The complexity of deleting a node when its predecessor is already known is:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Answer: B

E. Linked Representation of Stack and Queue

51. A stack follows which principle?

- A. FIFO
- B. LIFO
- C. Random access
- D. Priority-based access

Answer: B

52. A queue follows which principle?

- A. LIFO
- B. FIFO
- C. FILO only
- D. Random access

Answer: B

53. In a linked implementation of a stack, insertion and deletion are normally performed at:

- A. The end only
- B. The beginning/head
- C. The middle
- D. Any random node

Answer: B

54. Push operation in a linked stack has complexity:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Answer: C

55. Pop operation in a linked stack has complexity:

- A. $O(n)$
- B. $O(1)$
- C. $O(n^2)$
- D. $O(\log n)$

Answer: B

56. In a linked stack, the top pointer generally points to:

- A. Last inserted node
- B. First inserted node
- C. Middle node
- D. NULL always

Answer: A

57. In a linked queue, the two commonly maintained pointers are:

- A. Head and middle
- B. Front and rear
- C. Top and bottom
- D. Previous and current

Answer: B

58. Enqueue operation adds an element at:

- A. Front
- B. Rear
- C. Middle
- D. Top

Answer: B

59. Dequeue operation removes an element from:

- A. Rear
- B. Front
- C. Middle
- D. Top

Answer: B

60. With both front and rear pointers, enqueue in a linked queue can be:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Answer: C

61. With a front pointer, dequeue from a linked queue can be:

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log n)$

Answer: A

62. When the last element is removed from a linked queue:

- A. Front becomes NULL but rear remains valid
- B. Rear becomes NULL but front remains valid
- C. Both front and rear should become NULL
- D. Neither changes

Answer: C

63. A linked queue is useful because:

- A. Its size can dynamically grow
- B. It always has fixed capacity
- C. It requires contiguous memory
- D. It does not require pointers

Answer: A

64. Which operation is equivalent to inserting an element into a stack?

- A. Enqueue
- B. Push

- C. Dequeue
- D. Search

Answer: B

65. Which operation is equivalent to removing an element from a stack?

- A. Pop
- B. Push
- C. Enqueue
- D. Insert

Answer: A

F. Header Nodes

66. A header node is:

- A. Always the last node
- B. A special node used at the beginning of a linked list
- C. A deleted node
- D. A NULL node

Answer: B

67. A header node may be used to:

- A. Simplify list operations
- B. Increase array size
- C. Eliminate all pointers
- D. Sort automatically

Answer: A

68. In a header-node linked list, the header node generally:

- A. Stores user data only
- B. Is a special/sentinel node
- C. Must always be deleted first
- D. Is always the last node

Answer: B

69. One advantage of a header node is:

- A. It can reduce special cases during insertion/deletion
- B. It eliminates memory usage
- C. It provides random access
- D. It makes every operation $O(1)$

Answer: A

70. A header node is also commonly called a:

- A. Sentinel node
- B. Terminal node
- C. Data node
- D. Rear node

Answer: A

G. Doubly Linked Lists

71. A doubly linked list node generally contains:

- A. Data and one link
- B. Data and two links
- C. Two data fields and one link
- D. Only two links

Answer: B

72. The two links in a doubly linked list point to:

- A. Next and previous nodes
- B. Head and tail only
- C. Two next nodes
- D. Two previous nodes

Answer: A

73. A typical doubly linked list node has:

- A. prev, data, next
- B. data, next only
- C. head, data
- D. prev, head

Answer: A

74. The previous pointer of the first node normally contains:

- A. Address of last node
- B. NULL
- C. Address of second node
- D. Head

Answer: B

75. The next pointer of the last node normally contains:

- A. NULL
- B. Head
- C. Previous node
- D. First node

Answer: A

76. A major advantage of a doubly linked list is:

- A. It requires no extra memory
- B. Bidirectional traversal is possible
- C. It provides direct indexing
- D. It eliminates NULL pointers

Answer: B

77. Compared with a singly linked list, a doubly linked list requires:

- A. Less memory
- B. More memory
- C. No dynamic memory
- D. Fixed memory

Answer: B

78. The time complexity of traversal of a doubly linked list with n nodes is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

79. If a node in a doubly linked list is already known, deletion can generally be performed in:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Answer: B

80. To delete node X from a doubly linked list, which links may need updating?

- A. Only X's next
- B. Only X's previous
- C. Both neighboring nodes' links
- D. Head only

Answer: C

81. To insert node X before node Y in a doubly linked list, which pointers may need modification?

- A. Only X
- B. X and its neighboring nodes
- C. Only head
- D. Only tail

Answer: B

82. Doubly linked lists make it easier to:

- A. Move backward through the list
- B. Perform binary search
- C. Access arbitrary elements in $O(1)$
- D. Eliminate dynamic allocation

Answer: A

83. Which is a disadvantage of a doubly linked list?

- A. Cannot traverse forward
- B. Extra memory for previous links
- C. Cannot insert nodes
- D. Cannot delete nodes

Answer: B

84. If `current->prev` is NULL, `current` is usually:

- A. Last node
- B. First node
- C. Middle node
- D. Header only

Answer: B

85. If `current->next` is NULL, `current` is usually:

- A. First node
- B. Last node
- C. Middle node
- D. Header

Answer: B

H. Circular Linked Lists

86. In a circular singly linked list, the last node points to:

- A. NULL
- B. Itself always
- C. The first node
- D. Previous node

Answer: C

87. A circular linked list differs from a standard singly linked list because:

- A. It has no data
- B. The last node links back to the first node
- C. It has two previous pointers
- D. It cannot be traversed

Answer: B

88. In a circular linked list, traversal should NOT normally use:

- A. `current = current->next`
- B. Starting from head
- C. `current == NULL` as the only termination condition
- D. A condition based on returning to the starting node

Answer: C

89. A suitable termination condition for circular-list traversal is:

- A. `current == NULL`
- B. `current == head` after at least one traversal
- C. `current->next == NULL`
- D. `current == 0`

Answer: B

90. If a circular linked list contains one node, that node's next pointer points to:

- A. NULL
- B. Itself
- C. A random node
- D. Previous node

Answer: B

91. Circular linked lists are particularly useful for:

- A. Round-robin scheduling
- B. Binary search
- C. Static arrays
- D. Recursion only

Answer: A

92. If a circular singly linked list maintains only a tail pointer, the first node can be accessed using:

- A. `tail`
- B. `tail->next`
- C. `tail->previous`
- D. NULL

Answer: B

93. In a circular singly linked list with a tail pointer, insertion at the beginning can be performed in:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Answer: C

94. In a circular singly linked list with a tail pointer, insertion at the end can be:

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log n)$

Answer: A

95. In a circular linked list, the last node can be identified by:

- A. `node->next == NULL`
- B. `node->next == head`
- C. `node->prev == NULL`
- D. `node == NULL`

Answer: B

I. Complexity and Algorithm Analysis

96. What is the complexity of traversing any linked list containing n nodes?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

97. What is the complexity of inserting at the beginning of a singly linked list?

- A. $O(n)$
- B. $O(n^2)$
- C. $O(\log n)$
- D. $O(1)$

Answer: D

98. What is the complexity of searching for an element in an unsorted linked list in the worst case?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C

99. Which linked-list operation can be $O(1)$ when the required node/predecessor is already known?

- A. Insertion/deletion
- B. Searching an unknown value
- C. Finding the kth node
- D. Complete traversal

Answer: A

100. Which statement correctly summarizes linked-list complexity?

- A. Random access is $O(1)$ and insertion is always $O(n)$
- B. Traversal/search are generally $O(n)$, while insertion/deletion can be $O(1)$ when the relevant position/node is known
- C. Every linked-list operation is $O(\log n)$
- D. Every linked-list operation is $O(1)$

Answer: B

Quick Complexity Revision Table

Operation	Singly Linked List	Doubly Linked List	Circular Linked List
Access kth node	$O(n)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$
Traverse	$O(n)$	$O(n)$	$O(n)$
Insert at beginning	$O(1)$	$O(1)$	$O(1)^*$
Insert at end	$O(n)$ / $O(1)^{**}$	$O(n)$ / $O(1)^{**}$	$O(1)^*$

Operation	Singly Linked List	Doubly Linked List	Circular Linked List
Delete beginning	$O(1)$	$O(1)$	$O(1)$
Delete known node	$O(n) / O(1)^{***}$	$O(1)$	$O(1)^{***}$
Delete end	$O(n)$	$O(1)$ with tail	$O(n) / O(1)^{**}$

Notes:

- * Assuming an appropriate tail/head pointer is maintained.
- ** $O(1)$ when a tail pointer is maintained.
- *** $O(1)$ when the required node/predecessor is already available; otherwise locating it can take $O(n)$.

Most important exam points

Remember these:

- **Singly linked list:** data + next
- **Doubly linked list:** prev + data + next
- **Circular linked list:** last node points to first node
- **Stack:** LIFO
- **Queue:** FIFO
- **Push/Pop in linked stack:** $O(1)$
- **Enqueue/Dequeue in linked queue:** $O(1)$ with appropriate pointers
- **Traversal:** $O(n)$
- **Search:** $O(n)$ worst case
- **Random access:** $O(n)$, unlike arrays' $O(1)$
- **Insert at beginning:** $O(1)$
- **Delete at beginning:** $O(1)$
- **Header node:** special/sentinel node used to simplify operations
- **Doubly linked list:** supports forward and backward traversal
- **Circular list:** does not terminate with NULL at the last node.